

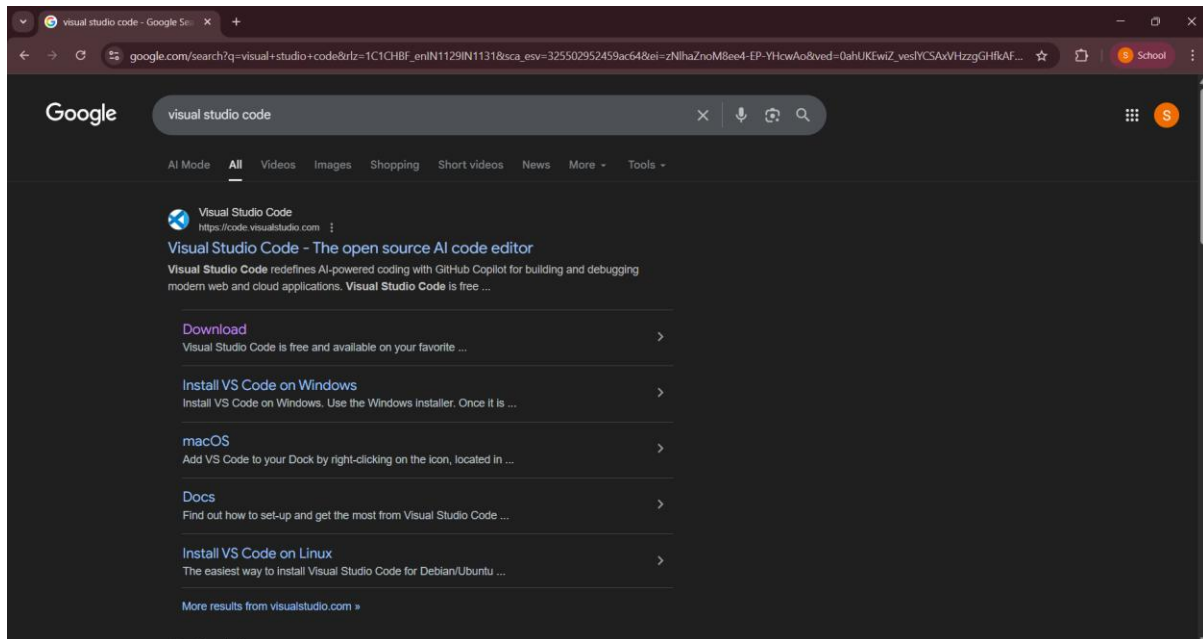
School of Computer Science and Artificial Intelligence

Lab Assignment 1.5

Program : B. Tech (CSE)
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : III
Academic Session : 2025-2026
Name of Student : Sankeerthana
Enrollment No. : 2403A51L41
Batch No. : 52
Date : 09-10-2026

Task 0

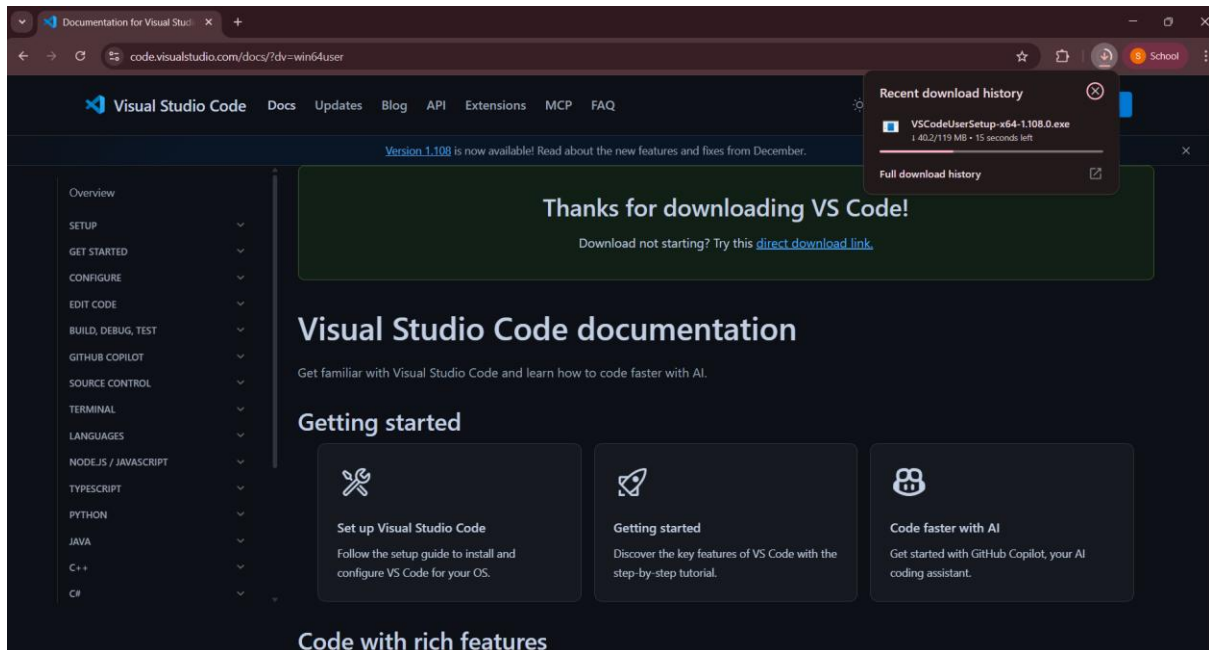
Search the Visual Studio Code in the browser. Then click on the Download.



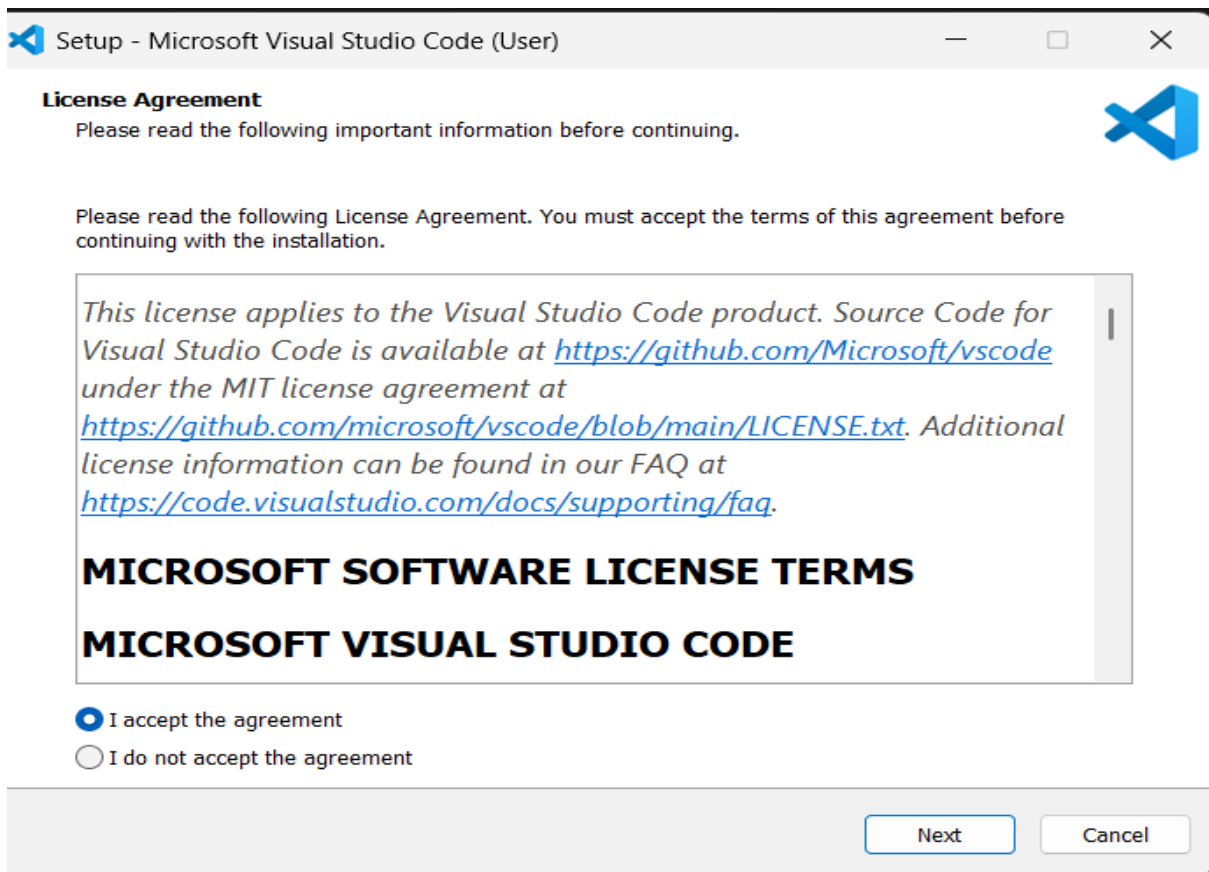
Click on the OS your desktop or laptop



A download .exe file you can see.

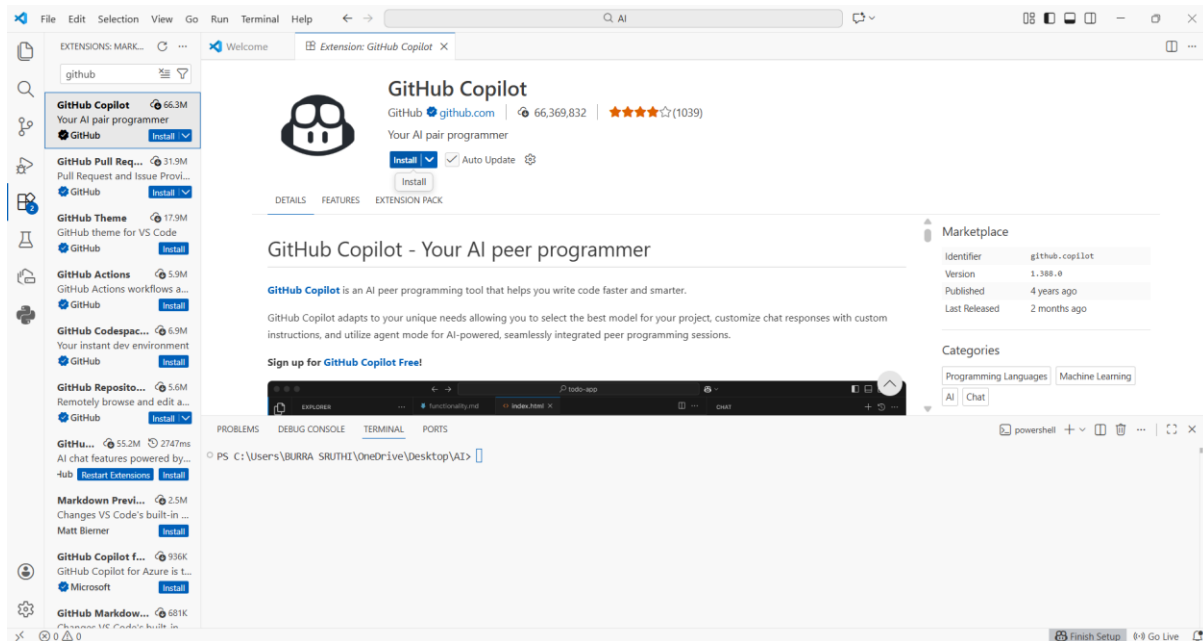


After the downloading the .exe file completed. Click on the .exe file then you will see the below window . Then click on the accept the agreement click next.

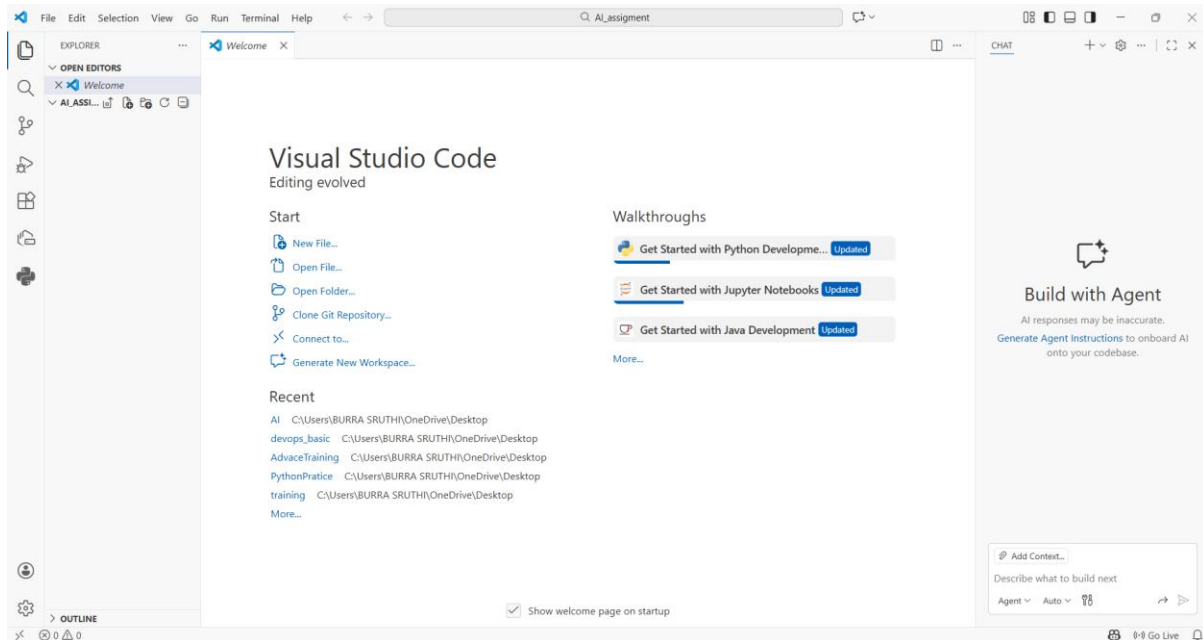


After that click next and then click on install. After the completion of download. Open the vs code

Click on the extension then search for GitHub Copilot. Click on the install



After the installation completed . sign in into the github copilot using the account of github. Create a new folder on desktop open it in the visual studio.

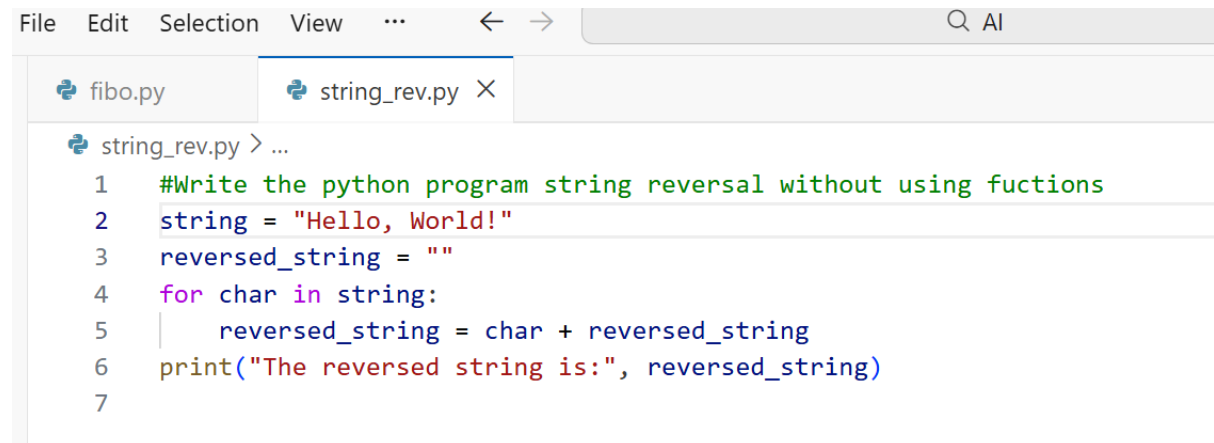


Create a new python file

Task1

Prompt: Write the python program string reversal without using functions

Code:



```
File Edit Selection View ... < > Q AI
fibonacci.py string_rev.py X
string_rev.py > ...
1 #Write the python program string reversal without using fuctions
2 string = "Hello, World!"
3 reversed_string = ""
4 for char in string:
5     reversed_string = char + reversed_string
6 print("The reversed string is:", reversed_string)
7
```

Output:

The reversed string is: !dlrow ,olleH

Explanation:

- In the above code the string is reversed by using for loop(iteration)
- The program initializes an empty string 'reversed_string' and iterates through each character in the original string.
- In each iteration, it prepends the current character to 'reversed_string', effectively reversing the order of characters.

Task2

Prompt: Write a python program to reverse a string without using functions and optimized way

Code:

```
string = "Hello, World!"
reversed_string = string[::-1]
print("The reversed string is:", reversed_string)
```

Output:

The reversed string is: !dlrow ,olleH

Explanation:

- The program uses Python's slicing feature to reverse the string in a single step.
- Slicing is used for to optimize the code

Task3

Prompt: Write a python program to reverse a string using recursion

Code:

```
def reverse_string_recursive(s):
    if len(s) == 0:
        return s
    else:
        return s[-1] + reverse_string_recursive(s[:-1])
string = "Hello, World!"
reversed_string = reverse_string_recursive(string)
print("The reversed string is:", reversed_string)
```

Output:

```
The reversed string is: !dlrow ,olleH
```

Explanation:

- The function 'reverse_string_recursive' takes a string 's' as input.
- It checks if the string is empty; if so, it returns the empty string.
- Otherwise, it concatenates the last character of the string with the result of reversing the substring
- that excludes the last character, effectively reversing the string recursively.

Task4

Prompt: write a python program to reverse a string with and without function

Code:

```
#without function
string = "Hello, World!"
reversed_string = ""
for char in string:
    reversed_string = char + reversed_string
print("Reversed string without function:", reversed_string)
#with function
def reverse_string(s):
    return s[::-1]
input_string = "Hello, World!"
reversed_str = reverse_string(input_string)
print("Reversed string with function:", reversed_str)
```

Output:

```
Reversed string without function: !dlrow ,olleH
Reversed string with function: !dlrow ,olleH
```

Explanation:

In the first part of the code, we reverse a string without using a function by iterating through each character of the string and prepending it to a new string.

In the second part, we define a function `reverse_string` that takes a string as input and returns the reversed string using slicing. We then call this function with an input string and print the result.

Comparative Analysis Table

1. Code Clarity:

- **Without functions:** The code is straightforward for small tasks but can become cluttered and harder to read as complexity increases.
- **With functions:** The code is more organized and easier to understand, as the logic is encapsulated within a function.

2. Reusability:

- **Without functions:** The code is not reusable; you would need to rewrite it each time you want to reverse a string.
- **With functions:** The function can be reused multiple times with different inputs, promoting code reuse.

3. Debugging Ease:

- **Without functions:** Debugging can be more challenging as the logic is not separated; any issues would require examining the entire code block.
- **With functions:** Debugging is easier since you can isolate the function and test it independently.

4. Suitability for Large-Scale Applications:

- **Without functions:** Not suitable for large-scale applications due to lack of modularity and maintainability.
- **With functions:** More suitable for large-scale applications as it promotes modularity, maintainability, and scalability.

Task 5

Prompt: Write a Python program to reverse a string using iteration and recursion

Code:

```
# Using iteration
def reverse_string_iterative(s):
    reversed_string = ""
    for char in s:
        reversed_string = char + reversed_string
    return reversed_string

# Using recursion
def reverse_string_recursive(s):
    if len(s) == 0:
        return s
    else:
        return s[-1] + reverse_string_recursive(s[:-1])

# Test the functions
input_string = "Hello, World!"
reversed_iterative = reverse_string_iterative(input_string)
reversed_recursive = reverse_string_recursive(input_string)
print("Reversed string using iteration:", reversed_iterative)
print("Reversed string using recursion:", reversed_recursive)
```

Output

```
Reversed string using iteration: !dlrow ,olleH
Reversed string using recursion: !dlrow ,olleH
```

Explanation

The iterative function builds the reversed string by prepending each character to a new string. The recursive function takes the last character of the string and appends it to the reversed version of the rest of the string.

Comparison:

1.Execution flow:

The iterative approach uses a loop to build the reversed string character by character. The recursive approach breaks the problem down into smaller subproblems, reversing the string by taking the last character and appending it to the reversed version of the rest of the string.

2.Time complexity:

Both approaches have a time complexity of $O(n)$, where n is the length of the string

3.Performance for large inputs:

The iterative approach is generally more efficient for large inputs as it avoids the overhead of multiple function calls associated with recursion.

The recursive approach can lead to a stack overflow for very large strings due to deep recursion.

4. When each approach is appropriate:

The iterative approach is suitable for most practical applications, especially when dealing with large strings.

The recursive approach can be more elegant and easier to understand for small strings or educational purposes, but should be used with caution for large inputs.